50 Advanced Single-Table SQL Interview Questions

Based on the Farmer's Market Database

Introduction

This document contains 50 advanced, scenario-based SQL questions designed for interview practice. The questions are based on a farmer's market database schema (using the provided CSV files as a reference). A key constraint is that **no JOINS are to be used**; every answer must come from querying a single table. The questions focus on aggregations (`GROUP BY`, `HAVING`, `SUM`, `AVG`, `COUNT`), conditional logic (`CASE`), derived columns, string matching (`LIKE`), and the logical order of query execution.

Table: `customer_purchases`

Question 1

We need to calculate the total revenue (defined as `quantity * cost_to_customer_per_qty`) generated by each `customer_id`. How would you write this query?

Answer.

```sql
SELECT
    customer_id,
    SUM(quantity * cost_to_customer_per_qty) AS total_revenue
FROM
    customer_purchases
GROUP BY
    customer_id
ORDER BY
    total_revenue DESC;
```

Question 2

Building on the previous question, how would you modify the query to show *only* customers who have spent more than $500 in total?

Answer.

```sql
SELECT
    customer_id,
    SUM(quantity * cost_to_customer_per_qty) AS total_revenue
FROM
    customer_purchases
GROUP BY
    customer_id
HAVING
    SUM(quantity * cost_to_customer_per_qty) > 500
ORDER BY
    total_revenue DESC;
```

Question 3

Management wants to analyze sales by time of day. How can you find the total `quantity` of items sold during 'Morning' (before '12:00:00') and 'Afternoon' ('12:00:00' or later) periods for each `market_date`?

Answer.

```sql
SELECT
    market_date,
    SUM(CASE
            WHEN transaction_time < '12:00:00' THEN quantity
            ELSE 0
        END) AS morning_quantity_sold,
    SUM(CASE
            WHEN transaction_time >= '12:00:00' THEN quantity
            ELSE 0
        END) AS afternoon_quantity_sold
FROM
    customer_purchases
GROUP BY
    market_date
ORDER BY
    market_date;
```

Question 4

What is the average `quantity` per transaction (i.e., per row) for each `product_id`?

Answer.

```sql
SELECT
    product_id,
    AVG(quantity) AS avg_quantity_per_transaction
FROM
    customer_purchases
GROUP BY
    product_id;
```

Question 5

For each `vendor_id`, find the highest and lowest `cost_to_customer_per_qty` they have ever charged for any product.

Answer.

```sql
SELECT
    vendor_id,
    MAX(cost_to_customer_per_qty) AS highest_price,
    MIN(cost_to_customer_per_qty) AS lowest_price
FROM
    customer_purchases
GROUP BY
    vendor_id;
```

Question 6

How many *unique customers* has each `vendor_id` sold to?

Answer.

```sql
SELECT
    vendor_id,
    COUNT(DISTINCT customer_id) AS unique_customer_count
FROM
    customer_purchases
GROUP BY
    vendor_id;
```

Question 7

Find all `product_id`s where the *average* quantity sold per transaction is greater than 2, but *only* for products that have been sold at least 10 times (i.e., appear in at least 10 transactions).

Answer.

```sql
SELECT
    product_id,
    AVG(quantity) AS avg_quantity,
    COUNT(*) AS transaction_count
FROM
    customer_purchases
GROUP BY
    product_id
HAVING
    AVG(quantity) > 2 AND COUNT(*) >= 10;
```

Question 8

What was the total revenue (quantity * cost) per day for July 2019? (e.g., `market_date` between '2019-07-01' and '2019-07-31').

Answer.

```sql
SELECT
    market_date,
    SUM(quantity * cost_to_customer_per_qty) AS daily_revenue
FROM
    customer_purchases
WHERE
    market_date >= '2019-07-01' AND market_date <= '2019-07-31'
GROUP BY
    market_date
ORDER BY
    market_date;
```

Question 9

Create a report that lists every transaction `product_id` and `quantity`, but adds a derived column named 'Transaction_Size'. This column should be 'Small' if `quantity` < 1, 'Medium' if `quantity` is between 1 and 5 (inclusive), and 'Large' if `quantity` > 5.

Answer.

```sql
SELECT
    product_id,
    quantity,
    CASE
        WHEN quantity < 1 THEN 'Small'
        WHEN quantity >= 1 AND quantity <= 5 THEN 'Medium'
        WHEN quantity > 5 THEN 'Large'
    END AS Transaction_Size
FROM
    customer_purchases;
```

Question 10

For `customer_id` 4, how many 'Small', 'Medium', and 'Large' transactions (using the logic from Q9) did they make in total?

Answer.

```sql
SELECT
    customer_id,
    COUNT(CASE WHEN quantity < 1 THEN 1 END) AS small_transactions,
    COUNT(CASE WHEN quantity >= 1 AND quantity <= 5 THEN 1 END) AS
    medium_transactions,
    COUNT(CASE WHEN quantity > 5 THEN 1 END) AS large_transactions
FROM
    customer_purchases
WHERE
    customer_id = 4
GROUP BY
    customer_id;
```

Question 11

We want to identify 'high-volume' products. Find all `product_id`s that have an *average* `quantity` per transaction greater than 2, AND have *never* been sold for a `cost_to_customer_per_qty` less than $1.00.

Answer.

```sql
SELECT
    product_id
FROM
    customer_purchases
GROUP BY
    product_id
HAVING
    AVG(quantity) > 2
    AND MIN(cost_to_customer_per_qty) >= 1.00;
```

Question 12

Find the `customer_id` who made the single largest purchase in terms of `total_cost` (quantity * cost) in a single transaction. Show the `customer_id` and that `total_cost`.

Answer.

```sql
SELECT
    customer_id,
    (quantity * cost_to_customer_per_qty) AS total_cost
FROM
    customer_purchases
ORDER BY
    total_cost DESC
LIMIT 1;
```

Question 13

We need a report for `vendor_id` 7. This report must show, for each `product_id` they sell, the total quantity sold, the total revenue, and a new column 'avg_revenue_per_unit' (calculated as total revenue / total quantity).

Answer.

```sql
SELECT
    product_id,
    SUM(quantity) AS total_quantity_sold,
    SUM(quantity * cost_to_customer_per_qty) AS total_revenue,
    SUM(quantity * cost_to_customer_per_qty) / SUM(quantity) AS
    avg_revenue_per_unit
FROM
    customer_purchases
WHERE
    vendor_id = 7
GROUP BY
    product_id;
```

Table: `vendor_inventory`

Question 14

What is the average `original_price` for each `product_id` listed in the inventory?

Answer.

```sql
SELECT
    product_id,
    AVG(original_price) AS avg_original_price
FROM
    vendor_inventory
GROUP BY
    product_id;
```

Question 15

How much total `quantity` of each `product_id` has each `vendor_id` brought to the market across all dates?

Answer.

```sql
SELECT
    vendor_id,
    product_id,
    SUM(quantity) AS total_inventory_quantity
FROM
    vendor_inventory
GROUP BY
    vendor_id, product_id
ORDER BY
    vendor_id, product_id;
```

Question 16

We want to categorize products by price. Show the `product_id` and a `price_category` ('Cheap' if `original_price` $<$ \$3, 'Moderate' if $<$ \$10, 'Expensive' otherwise) for all entries on '2019-08-21'.

Answer.

```sql
SELECT
    product_id,
    original_price,
    CASE
        WHEN original_price < 3 THEN 'Cheap'
        WHEN original_price < 10 THEN 'Moderate'
        ELSE 'Expensive'
    END AS price_category
FROM
    vendor_inventory
WHERE
    market_date = '2019-08-21';
```

Question 17

How many products offered by `vendor_id` 4 fall into the 'Cheap', 'Moderate', and 'Expensive' categories (using logic from Q16) across all market dates?

Answer.

```sql
SELECT
    vendor_id,
    COUNT(CASE WHEN original_price < 3 THEN 1 END) AS cheap_products,
    COUNT(CASE WHEN original_price >= 3 AND original_price < 10 THEN 1 END) AS
    moderate_products,
    COUNT(CASE WHEN original_price >= 10 THEN 1 END) AS expensive_products
FROM
    vendor_inventory
WHERE
    vendor_id = 4
GROUP BY
    vendor_id;
```

Question 18

Find all `product_id`s for which the total inventory `quantity` supplied across all dates by all vendors was less than 50 units.

Answer.

```sql
SELECT
    product_id,
    SUM(quantity) AS total_supplied
FROM
    vendor_inventory
GROUP BY
    product_id
HAVING
    SUM(quantity) < 50;
```

Question 19

For each `product_id`, find the highest and lowest `original_price` it has been listed for in the inventory.

Answer.

```sql
SELECT
    product_id,
    MAX(original_price) AS highest_listed_price,
    MIN(original_price) AS lowest_listed_price
FROM
    vendor_inventory
GROUP BY
    product_id;
```

Question 20

Find the `product_id` with the *most volatile* pricing. Volatility is defined as the difference between the `MAX(original_price)` and `MIN(original_price)`. Show the `product_id` and this price volatility.

Answer.

```sql
SELECT
    product_id,
    (MAX(original_price) - MIN(original_price)) AS price_volatility
FROM
    vendor_inventory
GROUP BY
    product_id
ORDER BY
    price_volatility DESC
LIMIT 1;
```

Table: `product`

Question 21

How many products have the word 'Organic' in their `product_name`?

Answer.

```sql
SELECT
    COUNT(*) AS organic_product_count
FROM
    product
WHERE
    product_name LIKE '%Organic%';
```

Question 22

Find all products whose `product_size` starts with '1' (e.g., '1 lb', '1.5 lbs', '10').

Answer.

```sql
SELECT
    product_name,
    product_size
FROM
    product
WHERE
    product_size LIKE '1%';
```

Question 23

Count how many products belong to each `product_category_id`.

Answer.

```sql
SELECT
    product_category_id,
    COUNT(*) AS product_count
FROM
    product
GROUP BY
    product_category_id;
```

Question 24

The `product_size` column has missing data, represented as `NULL`, empty strings (''), or whitespace (' '). Write a query that counts how many products have a 'Known Size' vs. an 'Unknown Size'.

Answer.

```sql
SELECT
    CASE
        WHEN product_size IS NULL OR TRIM(product_size) = '' OR product_size = '
    null'
        THEN 'Unknown Size'
        ELSE 'Known Size'
    END AS size_status,
    COUNT(*) AS product_count
FROM
    product
GROUP BY
    size_status;
```

(Note: The snippets show 'null' and ' ' as text, so checking for `TRIM` *and 'null' string is robust.)*

Question 25

Find all products that are sold by 'lbs' (pounds) based on the `product_qty_type` column.

Answer.

```sql
SELECT
    product_name,
    product_qty_type
FROM
    product
WHERE
    product_qty_type = 'lbs';
```

Question 26

The `product_name` column is messy. We want to count how many products are 'Peppers', 'Potatoes', or 'Tomatoes'. A product counts if its name contains *any* of those strings. Show the count for each group.

Answer.

```sql
SELECT
    SUM(CASE WHEN product_name LIKE '%Peppers%' THEN 1 ELSE 0 END) AS
    pepper_products,
    SUM(CASE WHEN product_name LIKE '%Potatoes%' THEN 1 ELSE 0 END) AS
    potato_products,
    SUM(CASE WHEN product_name LIKE '%Tomatoes%' THEN 1 ELSE 0 END) AS
    tomato_products
FROM
    product;
```

Table: `market_date_info`

Question 27

What is the average `market_max_temp` for each `market_season`?

Answer.

```sql
SELECT
    market_season,
    AVG(market_max_temp) AS avg_max_temp
FROM
    market_date_info
GROUP BY
    market_season;
```

Question 28

How many market days had rain (`market_rain_flag` $= 1$) and how many had snow (`market_snow_flag` $= 1$), grouped by `market_year`?

Answer.

```sql
SELECT
    market_year,
    SUM(market_rain_flag) AS total_rainy_days,
    SUM(market_snow_flag) AS total_snowy_days
FROM
    market_date_info
GROUP BY
    market_year
ORDER BY
    market_year;
```

Question 29

We want to know the distribution of market days. Count how many markets were held on 'Saturday', 'Wednesday', or 'Other'.

Answer.

```sql
SELECT
    CASE
        WHEN market_day = 'Saturday' THEN 'Saturday'
        WHEN market_day = 'Wednesday' THEN 'Wednesday'
        ELSE 'Other'
    END AS day_category,
    COUNT(*) AS market_day_count
FROM
    market_date_info
GROUP BY
    day_category;
```

Question 30

Find all `market_season`s where the average `market_min_temp` was below 40 degrees.

Answer.

```sql
SELECT
    market_season,
    AVG(market_min_temp) AS avg_min_temp
FROM
    market_date_info
GROUP BY
    market_season
HAVING
    AVG(market_min_temp) < 40;
```

Question 31

Create a 'Weather_Report' column for each `market_date`. It should say 'Snowy' if `market_snow_flag`=1, 'Rainy' if `market_rain_flag`=1, and 'Clear' otherwise. What is the logical flaw in this request and how would your CASE statement correctly handle a day that has both rain and snow?

Answer. *Logical Flaw:* A simple CASE statement will stop at the first WHEN that is true. If a day has both snow and rain, it might be miscategorized as 'Snowy' and the 'Rainy' aspect would be missed.

Correct Query:

```sql
SELECT
    market_date,
    market_rain_flag,
    market_snow_flag,
    CASE
        WHEN market_snow_flag = 1 AND market_rain_flag = 1 THEN 'Rain & Snow'
        WHEN market_snow_flag = 1 THEN 'Snowy'
        WHEN market_rain_flag = 1 THEN 'Rainy'
        ELSE 'Clear'
    END AS weather_report
FROM
    market_date_info
WHERE
    market_rain_flag = 1 OR market_snow_flag = 1; -- Example filter
```

(The CASE *statement must check for the combined condition first to be accurate.)*

Question 32

Categorize each `market_year` as 'Wet' or 'Dry'. A 'Wet' year is one where the *total* number of rainy days (`SUM(market_rain_flag)`) was greater than 15. Show the year and the category.

Answer.

```sql
SELECT
    market_year,
    SUM(market_rain_flag) AS total_rainy_days,
    CASE
        WHEN SUM(market_rain_flag) > 15 THEN 'Wet'
        ELSE 'Dry'
    END AS year_category
FROM
    market_date_info
GROUP BY
    market_year;
```

Table: `customer`

Question 33

How many customers have a last name that ends in 's'?

Answer.

```sql
SELECT
    COUNT(*) AS customer_count
FROM
    customer
WHERE
    customer_last_name LIKE '%s';
```

Question 34

How many customers are in each `customer_zip` code?

Answer.

```sql
SELECT
    customer_zip,
    COUNT(*) AS customer_count
FROM
    customer
GROUP BY
    customer_zip
ORDER BY
    customer_count DESC;
```

Question 35

A user wants to find all customers from zip code '2280' (e.g., '22801', '22802'). They write `SELECT * FROM customer WHERE customer_zip LIKE '2280_';`. What is wrong with this query and how would you fix it?

Answer. The _ (underscore) character in `LIKE` is a wildcard for *exactly one* character. The zip code '22801' has 5 characters, so `'2280_'` would match it. However, if there was a zip '2280', it would not match, and a zip '228012' (6 chars) would also not match.

The correct query to find all zip codes *starting with* '2280' is:

```sql
SELECT
    customer_id,
    customer_zip
FROM
    customer
WHERE
    customer_zip LIKE '2280%';
```

Table: `vendor`

Question 36

Find all vendors whose `vendor_name` includes an ampersand ('&').

Answer.

```sql
SELECT
    vendor_id,
    vendor_name
FROM
    vendor
WHERE
    vendor_name LIKE '%&%';
```

Question 37

What is the count of vendors for each `vendor_type`?

Answer.

```sql
SELECT
    vendor_type,
    COUNT(*) AS vendor_count
FROM
    vendor
GROUP BY
    vendor_type;
```

Question 38

How many vendors are owned by someone whose first name is 5 letters long and starts with 'M' (e.g., 'Maria', 'Marco')?

Answer.

```sql
SELECT
    COUNT(*) AS vendor_owner_count
FROM
    vendor
WHERE
    vendor_owner_first_name LIKE 'M____'; -- M followed by 4 underscores
```

Table: `booth`

Question 39

How many booths are there of each `booth_type`?

Answer.

```sql
SELECT
    booth_type,
    COUNT(*) AS booth_count
FROM
    booth
GROUP BY
    booth_type;
```

Question 40

From the `booth` table, find all booths whose `booth_description` mentions 'left end' or 'right end'.

Answer.

```sql
SELECT
    booth_number,
    booth_description
FROM
    booth
WHERE
    booth_description LIKE '%left end%'
    OR booth_description LIKE '%right end%';
```

Question 41

Re-classify the booths based on their description. If the description mentions 'center door', classify it as 'Entrance'. If it mentions 'left end' or 'right end', classify it as 'Endcap'. If it mentions 'Back', classify it as 'Back Wall'. Otherwise, classify it as 'Other'. Count how many booths fall into each new category.

Answer.

```sql
SELECT
    CASE
        WHEN booth_description LIKE '%center door%' THEN 'Entrance'
        WHEN booth_description LIKE '%left end%' OR booth_description LIKE '%right
    end%' THEN 'Endcap'
        WHEN booth_description LIKE '%Back%' THEN 'Back Wall'
        ELSE 'Other'
    END AS booth_category,
    COUNT(*) AS category_count
FROM
    booth
GROUP BY
    booth_category;
```

Table: `vendor_booth_assignments`

Question 42

Using `vendor_booth_assignments`, find out how many *times* each `vendor_id` has been assigned a booth (in total, across all dates).

Answer.

```sql
SELECT
    vendor_id,
    COUNT(*) AS total_assignments
FROM
    vendor_booth_assignments
GROUP BY
    vendor_id
ORDER BY
    total_assignments DESC;
```

Question 43

Using `vendor_booth_assignments`, find out how many *different* booths (`booth_number`) each `vendor_id` has used.

Answer.

```sql
SELECT
    vendor_id,
    COUNT(DISTINCT booth_number) AS unique_booths_used
FROM
    vendor_booth_assignments
GROUP BY
    vendor_id
ORDER BY
    unique_booths_used DESC;
```

Question 44

From `vendor_booth_assignments`, identify any `booth_number` that has been used by more than 3 *different vendors* over time.

Answer.

```sql
SELECT
    booth_number,
    COUNT(DISTINCT vendor_id) AS unique_vendor_count
FROM
    vendor_booth_assignments
GROUP BY
    booth_number
HAVING
    COUNT(DISTINCT vendor_id) > 3;
```

Question 45

From `vendor_booth_assignments`, find the `vendor_id` who has been assigned to booth `11` the most frequently.

Answer.

```sql
SELECT
    vendor_id,
    COUNT(*) AS assignment_count
FROM
    vendor_booth_assignments
WHERE
    booth_number = 11
GROUP BY
    vendor_id
ORDER BY
    assignment_count DESC
LIMIT 1;
```

Logical Order of Execution Questions

Question 46

A user wants to find the total revenue from `customer_purchases` for `customer_id` 10, but only for days where their *total* spending *on that day* was over $20. Explain why the following query fails and provide the correct one.

Failing Query:

```sql
SELECT market_date, SUM(quantity * cost_to_customer_per_qty)
FROM customer_purchases
WHERE customer_id = 10
  AND SUM(quantity * cost_to_customer_per_qty) > 20 -- Fails here
GROUP BY market_date;
```

Answer. *Explanation:* The query fails because the WHERE clause is processed *before* the GROUP BY clause and any aggregate functions (like SUM()). The WHERE clause filters individual rows. You cannot use an aggregate function in the WHERE clause.

To filter the *results* of an aggregation, you must use the HAVING clause, which is processed *after* GROUP BY.

Correct Query:

```sql
SELECT
    market_date,
    SUM(quantity * cost_to_customer_per_qty) AS daily_total
FROM
    customer_purchases
WHERE
    customer_id = 10 -- Filter rows first
GROUP BY
    market_date
HAVING
    SUM(quantity * cost_to_customer_per_qty) > 20; -- Filter groups next
```

Question 47

A user tries to create a derived column `total_cost` and filter on it in the WHERE clause. Why does this query fail, and how can it be fixed?

Failing Query:

```sql
SELECT
    product_id,
    quantity,
    (quantity * cost_to_customer_per_qty) AS total_cost
FROM
    customer_purchases
WHERE
    total_cost > 10; -- Fails here
```

Answer. *Explanation:* This fails because of the logical order of operations in SQL. The SELECT clause (which defines the alias `total_cost`) is processed *after* the WHERE clause. When the WHERE clause is being evaluated, the alias `total_cost` does not yet exist.

Fix (Option 1: Repeat Logic):

```sql
SELECT
    product_id,
    quantity,
    (quantity * cost_to_customer_per_qty) AS total_cost
FROM
```

```
    customer_purchases
WHERE
    (quantity * cost_to_customer_per_qty) > 10;
```

(Note: In many SQL dialects, you could also use a subquery or Common Table Expression (CTE), but that is beyond the scope of a single-table query structure.)

Question 48

A user wants a list of all unique `customer_id`s from the `customer_purchases` table. They write: `SELECT customer_id FROM customer_purchases GROUP BY customer_id;`

A colleague suggests: `SELECT DISTINCT customer_id FROM customer_purchases;`

Is the *result* of these two queries different? Which is likely to be more computationally efficient and why?

Answer. *Result:* The result set for these two queries will be **identical**. Both will return a unique list of `customer_id`s that appear in the table.

Efficiency: `SELECT DISTINCT` is generally more efficient for this specific purpose.

- `GROUP BY` is designed to partition rows into groups so that aggregate functions (like `SUM`, `COUNT`) can be applied. It often involves sorting the entire dataset by the grouping key, which can be resource-intensive.
- `DISTINCT` is specifically designed to remove duplicate rows from the result set. While it also often uses a sort, its operation is more specialized and can be optimized more effectively by the database engine for just finding unique values.

Question 49

In the `product` table, if you run `SELECT AVG(product_category_id) FROM product;`, will the result always be the same as `SELECT SUM(product_category_id) / COUNT(product_category_id) FROM product;`?

What about `SELECT SUM(product_category_id) / COUNT(*) FROM product;`? Explain the difference, especially considering `NULL` values.

Answer.

- `AVG(col)` vs. `SUM(col) / COUNT(col)`: **Yes**, these two are equivalent. `AVG(col)`, `SUM(col)`, and `COUNT(col)` all **ignore `NULL` values**. The average is the sum of non-null values divided by the count of non-null values.
- `AVG(col)` vs. `SUM(col) / COUNT(*)`: **No**, these can be different. `COUNT(*)` counts **all rows** in the table, regardless of whether any column is `NULL`. If `product_category_id` has `NULL` values, `COUNT(*)` will be larger than `COUNT(product_category_id)`, which will make the denominator larger and the resulting average smaller.

Question 50

A user wants to find all products from the `product` table whose `product_name` contains 'Peppers' and are in `product_category_id` 1. They are unsure about the order of `WHERE` clause predicates.

Do these two queries produce a different result or have a different performance?

Query A:

```sql
SELECT * FROM product
WHERE product_name LIKE '%Peppers%'
  AND product_category_id = 1;
```

Query B:

```sql
SELECT * FROM product
WHERE product_category_id = 1
  AND product_name LIKE '%Peppers%';
```

Answer. *Result:* The result set will be **identical**. The SQL query optimizer is responsible for determining the most efficient way to execute the query, and the order of AND predicates in the WHERE clause does not change the logical outcome.

Performance: In a modern SQL database, the performance will likely be **identical or negligible**. The query optimizer will analyze the predicates and choose the most "selective" one to execute first.

- If `product_category_id` is indexed, the optimizer will almost certainly execute `product_category_id = 1` first, as this is much faster (an index lookup) than a full table scan required by `product_name LIKE '%Peppers%'` (a "leading wildcard" LIKE cannot use a standard index).
- The optimizer is smart enough to do this regardless of which order you write the predicates in.